

INFORME TÉCNICO

TurtleSim

Simulador de Robot Móvil con Control por Teclado y Visualización en Tiempo Real

Materia: Estándares Web

Institución: Universidad de Montemorelos

Integrantes: Edmila Tchisseke Coelho da Cruz Camisa

Fecha: 7 de mayo de 2026

Documento generado a partir del código fuente del proyecto.

1. Introducción

El presente informe documenta el desarrollo de TurtleSim, una aplicación web interactiva que simula el movimiento de un robot móvil en un plano cartesiano 2D controlado por el usuario mediante el teclado. El proyecto está inspirado en el simulador turtlesim del framework ROS (Robot Operating System), adaptado como una aplicación web ligera con Node.js y JavaScript puro.

La aplicación permite al usuario controlar en tiempo real la posición, orientación y velocidad de un robot representado visualmente en el plano, observar la trayectoria generada por sus movimientos, monitorear métricas de desplazamiento y consultar un historial de estados. Todo ello sin frameworks de frontend, utilizando únicamente HTML5, CSS3, JavaScript vanilla y Chart.js.

2. Objetivos

2.1 Objetivo General

Desarrollar una aplicación web de simulación de robot móvil que permita al usuario controlar su movimiento mediante el teclado, visualizando en tiempo real la trayectoria en un plano cartesiano, las velocidades y un historial de estados, aplicando conceptos de cinemática diferencial y desarrollo web.

2.2 Objetivos Específicos

- Implementar el control de movimiento del robot mediante eventos de teclado (flechas direccionales).
- Calcular y actualizar en tiempo real la posición (x, y) y orientación (θ) del robot usando cinemática diferencial.
- Visualizar la trayectoria del robot sobre un plano cartesiano con Chart.js mediante un plugin personalizado.
- Mostrar una gráfica de velocidades lineal y angular en tiempo real con ventana deslizante de 40 muestras.
- Registrar un historial de estados (posición, velocidad, pasos) con desplazamiento automático.
- Servir la aplicación mediante un servidor Express mínimo sin dependencias adicionales de frontend.

3. Tecnologías Utilizadas

Tecnología	Versión	Rol en el Proyecto
Node.js	LTS	Entorno de ejecución del servidor
Express.js	^5.2.1	Servidor web que sirve los archivos estáticos
HTML5 + CSS3	Nativo	Estructura y estilos de la interfaz (tema oscuro)
JavaScript (Vanilla)	ES2020+	Lógica de simulación, eventos y renderizado
Chart.js	CDN latest	Gráficas de trayectoria y velocidades en tiempo real

4. Arquitectura del Sistema

TurtleSim tiene una arquitectura extremadamente sencilla y ligera. El servidor Express (index.js) sirve únicamente el archivo index.html como contenido estático en el puerto 3003. Toda la lógica de simulación, el motor gráfico y el manejo de eventos viven íntegramente en el cliente dentro del archivo index.html, sin peticiones adicionales al servidor durante la ejecución.

4.1 Servidor Express (index.js)

El servidor tiene solo 6 líneas funcionales: importa Express, configura el middleware express.static() apuntando al directorio raíz del proyecto y levanta el servidor en el puerto 3003. No expone ninguna API REST ni procesa ningún dato — su único rol es entregar el archivo HTML al navegador.

```
app.use(express.static(path.join(__dirname)));
app.listen(3003, () => console.log("TurtleSim → http://localhost:3003"));
```

4.2 Motor de Simulación (index.html)

La simulación corre en un bucle setInterval de 50 ms (20 fps). En cada ciclo se leen las teclas presionadas, se actualizan las velocidades, se integran las ecuaciones de movimiento, se actualizan las gráficas y los indicadores del DOM. El estado completo del robot se mantiene en variables globales de JavaScript.

5. Cinemática del Robot

El modelo cinemático implementado corresponde a un robot diferencial simplificado con velocidad lineal y angular independientes. Las constantes de movimiento son:

Constante	Valor	Descripción
LINEAR_SPEED	8 unidades/ciclo	Desplazamiento por ciclo al avanzar/retroceder
ANGULAR_SPEED	0.05 rad/ciclo	Incremento de orientación por ciclo al girar
Intervalo	50 ms	Período del bucle de simulación (20 Hz)

5.1 Ecuaciones de movimiento

En cada ciclo de 50 ms se aplican las siguientes ecuaciones según la tecla presionada:

```
theta += velW // actualizar orientación
posX  += velV * cos(theta) // actualizar X
posY  += velV * sin(theta) // actualizar Y
```

Donde velV toma el valor LINEAR_SPEED (avanzar), -LINEAR_SPEED (retroceder) o 0 (girar/quieto), y velW toma ANGULAR_SPEED (izquierda), -ANGULAR_SPEED (derecha) o 0.

5.2 Control por teclado

Tecla	velV	velW	Efecto
↑ ArrowUp	+8	0	Avanzar en dirección θ actual
↓ ArrowDown	-8	0	Retroceder en dirección θ actual
← ArrowLeft	0	+0.05	Girar a la izquierda (CCW)
→ ArrowRight	0	-0.05	Girar a la derecha (CW)

6. Interfaz de Usuario

La interfaz presenta un tema oscuro con fondo #0d0d0d y tipografía monospace, inspirado en terminales de sistemas embebidos. Se organiza en los siguientes componentes:

6.1 Panel de Indicadores de Teclado

Fila de hints en la parte superior que muestra los controles disponibles con etiquetas estilo kbd, indicando la tecla y su acción correspondiente.

6.2 Tarjetas de Métricas

Tres tarjetas con borde izquierdo de color identificador: POSICIÓN (verde #00ff99) muestra x , y y θ en radianes; VELOCIDAD (naranja #EF9F27) muestra velocidad lineal v , angular ω y la dirección actual en texto; RECORRIDO (azul #378ADD) muestra la distancia total acumulada y el número de pasos realizados.

6.3 Gráfica de Velocidades en Tiempo Real

Gráfica de líneas con Chart.js que muestra las últimas 40 muestras de velocidad lineal (naranja) y angular (azul punteado). Tiene animación desactivada para garantizar actualización fluida a 20 Hz. La ventana deslizante elimina muestras antiguas para mantener el rendimiento.

6.4 Plano Cartesiano con Plugin de Robot

Gráfica de dispersión (scatter) con un plugin personalizado de Chart.js (robotPlugin) que dibuja sobre el canvas en cada ciclo. El plugin renderiza la estela de la trayectoria en verde, el cuerpo del robot como un cuadrado con borde verde y un punto blanco central, y un triángulo naranja como indicador de dirección. Los ejes del plano se reescalan automáticamente cada 20 pasos para seguir al robot.

6.5 Historial de Estados

Tabla con scroll vertical (máximo 130 px) que registra cada 5 pasos el estado del robot: número de paso, coordenadas x e y , orientación θ , velocidad lineal v y velocidad angular ω . Las entradas nuevas se insertan al inicio (prepend) para mostrar siempre el estado más reciente en la parte superior.

7. Estructura del Proyecto

Archivo	Descripción
index.html	Interfaz completa: estructura HTML, estilos CSS y lógica JS de simulación
index.js	Servidor Express mínimo que sirve los archivos estáticos en el puerto 3003
package.json	Metadatos del proyecto y dependencia de Express 5
node_modules/	Dependencias instaladas (no se sube al repositorio)

Nota: La carpeta node_modules/ no se incluye en el repositorio. Es generada automáticamente al ejecutar npm install y puede omitirse en el .gitignore.

8. Resultados

A continuación se presentan capturas de pantalla de TurtleSim en funcionamiento, mostrando el estado inicial de la aplicación y una sesión de control activa.

8.1 Estado inicial

Al abrir la aplicación, el robot aparece en el origen del plano cartesiano ($x: 0.00$, $y: 0.00$, $\theta: 0.0000$ rad) con todas las métricas en cero. El plano muestra el cuadrado del robot con la flecha naranja apuntando hacia la derecha (dirección positiva del eje X), listo para recibir comandos del teclado.

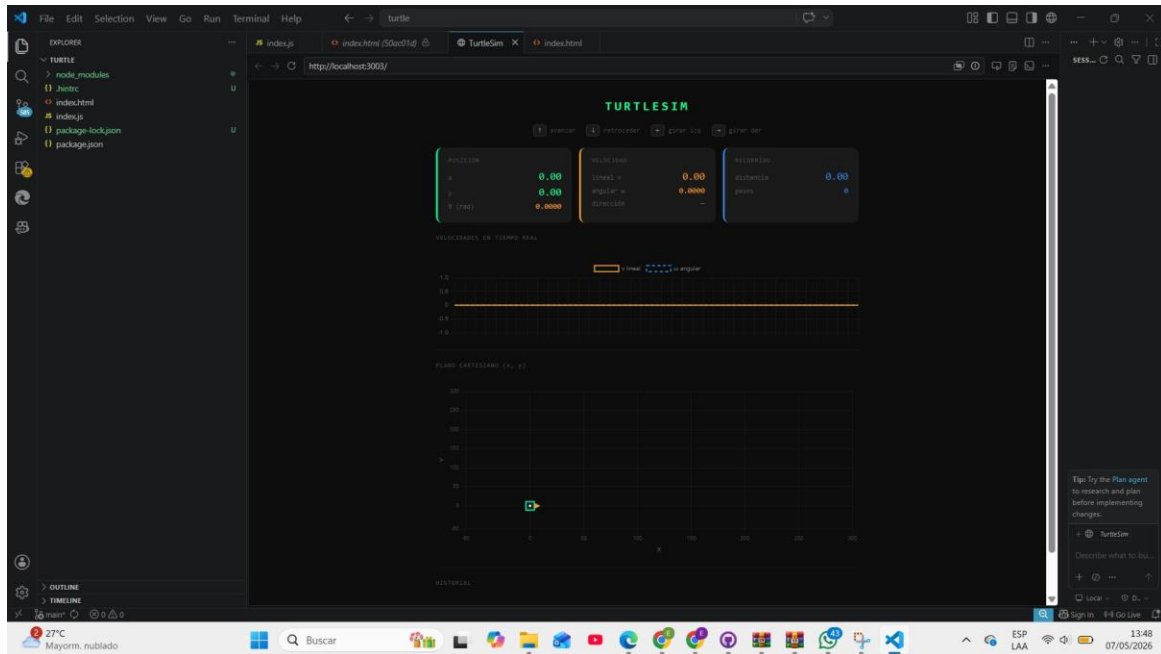


Figura 1. TurtleSim — Estado inicial del simulador con el robot en el origen (0, 0).

8.2 Robot en movimiento

Tras una sesión de control con las teclas de dirección, el robot alcanza la posición $x: 335.21$, $y: 181.75$ con una orientación de $\theta: 1.2500$ rad. La trayectoria verde dibujada en el plano refleja los cambios de dirección realizados por el usuario: avances rectos interrumpidos por giros. La distancia total acumulada es 480.00 unidades en 179 pasos, y la gráfica de velocidades confirma la actividad de movimiento.

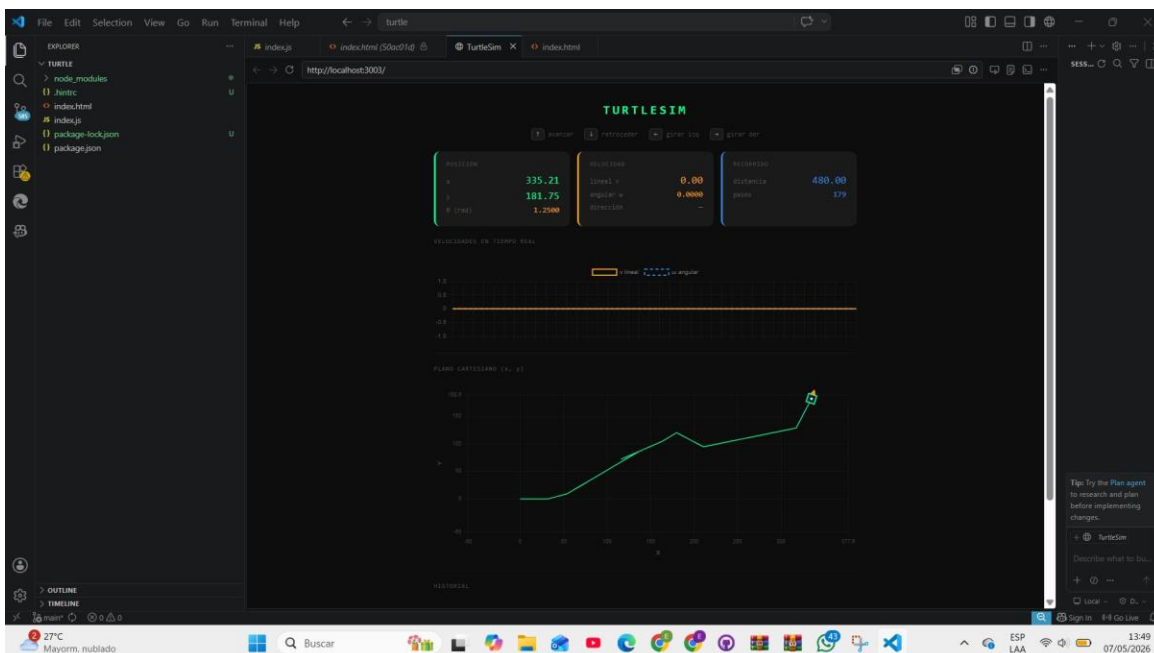


Figura 2. TurtleSim — Robot en movimiento con trayectoria generada por control de teclado.

9. Instalación y Ejecución

Requisitos previos: Node.js (versión LTS recomendada).

1. Clonar o descomprimir el proyecto en una carpeta local.
2. Instalar las dependencias (solo se necesita Express):

```
npm install
```

Nota: La carpeta node_modules/ no está incluida en el repositorio, por lo que este paso es obligatorio antes de ejecutar el proyecto por primera vez.

3. Iniciar el servidor:

```
node index.js
```

4. Abrir el navegador en:

```
http://localhost:3003
```

5. Hace click primero en la pantalla, para que el teclado responda.
6. Usar las teclas de flecha para controlar el robot:

Tecla	Acción
↑	Avanzar en la dirección actual del robot
↓	Retroceder en la dirección actual del robot
←	Girar a la izquierda (sentido antihorario)
→	Girar a la derecha (sentido horario)

10. Conclusiones

TurtleSim demuestra que es posible construir un simulador de robot móvil funcional y visualmente atractivo con tecnologías web mínimas: un servidor Express de 6 líneas, HTML5 nativo, CSS3 con variables y JavaScript vanilla sin frameworks. La ausencia de dependencias de frontend reduce la complejidad del proyecto y facilita su comprensión y mantenimiento.

La implementación del plugin personalizado de Chart.js para el renderizado del robot y su trayectoria sobre un canvas es un ejemplo práctico de extensión de librerías de visualización. El bucle de simulación a 20 Hz con `setInterval` garantiza una experiencia de control fluida y responsiva.

Como trabajo futuro se propone: agregar botones de control en pantalla para dispositivos móviles (touch), implementar un botón de reset para limpiar la trayectoria, añadir obstáculos en el plano, permitir configurar las constantes de velocidad desde la interfaz, y exportar la trayectoria generada como archivo CSV o imagen PNG.

11. Referencias

- Open Robotics. (2024). ROS turtlesim — ROS Wiki. <https://wiki.ros.org/turtlesim>
- Chart.js. (2024). Chart.js – Simple yet flexible JavaScript charting. <https://www.chartjs.org>
- Expressjs.com. (2024). Express – Fast, unopinionated web framework for Node.js. <https://expressjs.com>
- MDN Web Docs. (2024). KeyboardEvent – Web APIs. <https://developer.mozilla.org/es/docs/Web/API/KeyboardEvent>
- MDN Web Docs. (2024). Canvas API – Web APIs. https://developer.mozilla.org/es/docs/Web/API/Canvas_API
- Siegwart, R., Nourbakhsh, I. R., & Scaramuzza, D. (2011). Introduction to Autonomous Mobile Robots (2.a ed.). MIT Press.